Introduction to Haskell Programming

Input/Output

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

User interaction

- Till now ...

User interaction

- Till now ...
  - A **program** is a bunch of functions

User interaction

- Till now ...
  - A **program** is a bunch of functions
  - A function of type a → b produces a result of type b on an argument of type a

User interaction

- Till now ...
  - A **program** is a bunch of functions
  - A function of type `a → b` produces a result of type `b` on an argument of type `a`
  - The programs are run in `ghci` – by invoking a function on some arguments

User interaction

- Till now ...
  - A **program** is a bunch of functions
  - A function of type `a → b` produces a result of type `b` on an argument of type `a`
  - The programs are run in `ghci` – by invoking a function on some arguments
  - `ghci` automatically displays the result on the screen (provided it can be shown)

User interaction

- Till now ...
  - A **program** is a bunch of functions
  - A function of type `a → b` produces a result of type `b` on an argument of type `a`
  - The programs are run in `ghci` – by invoking a function on some arguments
  - `ghci` automatically displays the result on the screen (provided it can be shown)
- Can we execute programs outside `ghci`?

User interaction

- Till now ...
  - A **program** is a bunch of functions
  - A function of type `a → b` produces a result of type `b` on an argument of type `a`
  - The programs are run in `ghci` – by invoking a function on some arguments
  - `ghci` automatically displays the result on the screen (provided it can be shown)
- Can we execute programs outside `ghci`?
- How do we let the programs interact with users?

User interaction

- Till now ...
  - A **program** is a bunch of functions
  - A function of type `a → b` produces a result of type `b` on an argument of type `a`
  - The programs are run in `ghci` – by invoking a function on some arguments
  - `ghci` automatically displays the result on the screen (provided it can be shown)
- Can we execute programs outside `ghci`?
- How do we let the programs interact with users?
  - Accept user inputs midway through a program execution

User interaction

- Till now ...
  - A **program** is a bunch of functions
  - A function of type `a → b` produces a result of type `b` on an argument of type `a`
  - The programs are run in `ghci` – by invoking a function on some arguments
  - `ghci` automatically displays the result on the screen (provided it can be shown)
- Can we execute programs outside `ghci`?
- How do we let the programs interact with users?
  - Accept user inputs midway through a program execution
  - Print output and diagnostics on screen or to a file

Standalone programs and `main`

- Execution of a Haskell program starts with the function `main`

Standalone programs and `main`

- Execution of a Haskell program starts with the function `main`
- Every standalone Haskell program should have a `main` function

First program

- First compilable program

```
main = putStr "Hello, world!\n"
```

First program

- First compilable program

```
main = putStr "Hello, world!\n"
```

- Save this into a file named `hw.hs`

First program

- First compilable program

```
main = putStr "Hello, world!\n"
```

- Save this into a file named `hw.hs`
- Compile it using the command `ghc hw.hs`

First program

- First compilable program

```
main = putStr "Hello, world!\n"
```

- Save this into a file named `hw.hs`
- Compile it using the command `ghc hw.hs`
- This generates the files `hw.hi`, `hw.o` and `hw` (with **execute permissions**)

First program

- First compilable program

```
main = putStr "Hello, world!\n"
```

- Save this into a file named `hw.hs`
- Compile it using the command `ghc hw.hs`
- This generates the files `hw.hi`, `hw.o` and `hw` (with **execute permissions**)
- Run the executable using the command `./hw`

ghc

- `ghc` is the Glasgow Haskell Compiler

ghc

- `ghc` is the Glasgow Haskell Compiler
- `ghci` is the **interactive** version of the compiler

ghc

- `ghc` is the Glasgow Haskell Compiler
- `ghci` is the **interactive** version of the compiler
- One can view `ghci` as an **interpreter** or a **playground** in which to test programs

ghc

- `ghc` is the Glasgow Haskell Compiler
- `ghci` is the **interactive** version of the compiler
- One can view `ghci` as an **interpreter** or a **playground** in which to test programs
- Software intended for use by others is written as a standalone program, compiled using `ghc` and shipped

`ghc`

- Compiled versions of programs run much faster and use much less memory, compared to running them in `ghci`

`ghc`

- Compiled versions of programs run much faster and use much less memory, compared to running them in `ghci`
- Check out commonly used compiler options using `ghc --help`

ghc

- Compiled versions of programs run much faster and use much less memory, compared to running them in `ghci`
- Check out commonly used compiler options using `ghc --help`
- Use `ghc --show-options` to know all options (a huge list!)

ghc

- Compiled versions of programs run much faster and use much less memory, compared to running them in `ghci`
- Check out commonly used compiler options using `ghc --help`
- Use `ghc --show-options` to know all options (a huge list!)
- The **GHC Manual** is a comprehensive document about both `ghc` and `ghci`

`ghc`

- Compiled versions of programs run much faster and use much less memory, compared to running them in `ghci`
- Check out commonly used compiler options using `ghc --help`
- Use `ghc --show-options` to know all options (a huge list!)
- The **GHC Manual** is a comprehensive document about both `ghc` and `ghci`
  - Available at **this URL**!

Hello, world!

- The program

```
main = putStr "Hello, world!\n"
```

Hello, world!

- The program

```
main = putStr "Hello, world!\n"
```

- `putStr str` prints the string `str` on screen

Hello, world!

- The program

```
main = putStr "Hello, world!\n"
```

- `putStr str` prints the string `str` on screen
- Clearly `putStr` is of type `String → b`, for some `b`

Hello, world!

- The program

```
main = putStr "Hello, world!\n"
```

- `putStr str` prints the string `str` on screen
- Clearly `putStr` is of type `String → b`, for some `b`
- The return value is not used at all, so perhaps it returns nothing of significance

Hello, world!

- The program

```
main = putStr "Hello, world!\n"
```

- `putStr str` prints the string `str` on screen
- Clearly `putStr` is of type `String → b`, for some `b`
- The return value is not used at all, so perhaps it returns nothing of significance
- The type `()`, pronounced **unit**

Hello, world!

- The program

```
main = putStr "Hello, world!\n"
```

- `putStr str` prints the string `str` on screen
- Clearly `putStr` is of type `String → b`, for some `b`
- The return value is not used at all, so perhaps it returns nothing of significance
- The type `()`, pronounced **unit**
  - Consists of a single value, also denoted by `()`

Hello, world!

- The program

```
main = putStr "Hello, world!\n"
```

- `putStr str` prints the string `str` on screen
- Clearly `putStr` is of type `String → b`, for some `b`
- The return value is not used at all, so perhaps it returns nothing of significance
- The type `()`, pronounced **unit**
  - Consists of a single value, also denoted by `()`
  - Can be used to model **a dummy value (of a dummy type)**

Hello, world!

- The program

```
main = putStr "Hello, world!\n"
```

- `putStr str` prints the string `str` on screen
- Clearly `putStr` is of type `String → b`, for some `b`
- The return value is not used at all, so perhaps it returns nothing of significance
- The type `()`, pronounced **unit**
  - Consists of a single value, also denoted by `()`
  - Can be used to model **a dummy value (of a dummy type)**
- So is `String → ()` the type of `putStr`?

Hello, world!

- Is `putStr` of type `String → ()`?

Hello, world!

- Is `putStr` of type `String → ()`?
- **No**, because it does not return the value `()`!

Hello, world!

- Is `putStr` of type `String → ()`?
- **No**, because it does not return the value `()`!
- Further, how do we account for the side effect of printing something on screen?

Hello, world!

- Is `putStr` of type `String → ()`?
- **No**, because it does not return the value `()`!
- Further, how do we account for the side effect of printing something on screen?
- Checking the types in GHCi ...

```
ghci> :t putStr
    putStr :: String → IO ()

ghci> :t putStr "Hello, world!"
    putStr "Hello, world!" :: IO ()
```

`IO a`

- `IO` is a **type constructor**, just like `Maybe` or `[]`

`IO a`

- `IO` is a **type constructor**, just like `Maybe` or `[]`
  - If `a` is a type, then `Maybe a` and `[a]` are types

`IO a`

- `IO` is a **type constructor**, just like `Maybe` or `[]`
  - If `a` is a type, then `Maybe a` and `[a]` are types
  - Likewise, `IO a` is a type whenever `a` is a type

`IO a`

- `IO` is a **type constructor**, just like `Maybe` or `[]`
  - If `a` is a type, then `Maybe a` and `[a]` are types
  - Likewise, `IO a` is a type whenever `a` is a type
- Values of type `Maybe a`

`IO a`

- `IO` is a **type constructor**, just like `Maybe` or `[]`
  - If `a` is a type, then `Maybe a` and `[a]` are types
  - Likewise, `IO a` is a type whenever `a` is a type
- Values of type `Maybe a`
  - Either `Nothing` or `Just x` for `x :: a`

`IO a`

- `IO` is a **type constructor**, just like `Maybe` or `[]`
  - If `a` is a type, then `Maybe a` and `[a]` are types
  - Likewise, `IO a` is a type whenever `a` is a type
- Values of type `Maybe a`
  - Either `Nothing` or `Just x` for `x :: a`
  - `Nothing` and `Just` are **value constructors**

`IO a`

- `IO` is a **type constructor**, just like `Maybe` or `[]`
  - If `a` is a type, then `Maybe a` and `[a]` are types
  - Likewise, `IO a` is a type whenever `a` is a type
- Values of type `Maybe a`
  - Either `Nothing` or `Just x` for `x :: a`
  - `Nothing` and `Just` are **value constructors**
- Unlike other type constructors like `Maybe`, the internal structure and constructors of `IO` are not visible to the user

`IO a`

- `IO a` can be understood to be the type

```
RealWorld → (RealWorld,a)
```

`IO a`

- `IO a` can be understood to be the type

```
RealWorld → (RealWorld,a)
```

- So an object of type `IO a` is a function:

`IO a`

- `IO a` can be understood to be the type

```
RealWorld → (RealWorld,a)
```

- So an object of type `IO a` is a function:
  - Takes the current state of the real world as input

IO a

- `IO a` can be understood to be the type

```
RealWorld → (RealWorld,a)
```

- So an object of type `IO a` is a function:
  - Takes the current state of the real world as input
  - Produces a new state of the real world and a value of type `a`

`IO a`

- `IO a` can be understood to be the type

```
RealWorld → (RealWorld,a)
```

- So an object of type `IO a` is a function:
  - Takes the current state of the real world as input
  - Produces a new state of the real world and a value of type `a`
- In other words, objects of `IO a` consist of two things:

IO a

- `IO a` can be understood to be the type

```
RealWorld → (RealWorld,a)
```

- So an object of type `IO a` is a function:
  - Takes the current state of the real world as input
  - Produces a new state of the real world and a value of type a
- In other words, objects of `IO a` consist of two things:
  - A value of type a that can be extracted

IO a

- `IO a` can be understood to be the type

```
RealWorld → (RealWorld,a)
```

- So an object of type `IO a` is a function:
  - Takes the current state of the real world as input
  - Produces a new state of the real world and a value of type `a`
- In other words, objects of `IO a` consist of two things:
  - A value of type `a` that can be extracted
  - A side effect (the change in state of the world)

`IO` a and actions

- Technically, an object of type `IO a` is not a function

`IO a` and actions

- Technically, an object of type `IO a` is not a function
- It is an **action** of return type `IO a`

`IO a` and actions

- Technically, an object of type `IO a` is not a function
- It is an **action** of return type `IO a`
- An IO action produces a side effect **when its value is extracted**

`IO a` and actions

- Technically, an object of type `IO a` is not a function
- It is an **action** of return type `IO a`
- An IO action produces a side effect **when its value is extracted**
- Any function that produces a side effect will have return type `IO a`

`IO a` and actions

- Technically, an object of type `IO a` is not a function
- It is an **action** of return type `IO a`
- An IO action produces a side effect **when its value is extracted**
- Any function that produces a side effect will have return type `IO a`
- Just as we **compose functions**, we can **chain actions**

`IO a` and actions

- Technically, an object of type `IO a` is not a function
- It is an **action** of return type `IO a`
- An IO action produces a side effect **when its value is extracted**
- Any function that produces a side effect will have return type `IO a`
- Just as we **compose functions**, we can **chain actions**
- Pure functions and actions can be embedded inside actions ...

`IO a` and actions

- Technically, an object of type `IO a` is not a function
- It is an **action** of return type `IO a`
- An IO action produces a side effect **when its value is extracted**
- Any function that produces a side effect will have return type `IO a`
- Just as we **compose functions**, we can **chain actions**
- Pure functions and actions can be embedded inside actions ...
- ...but actions cannot be part of pure functions

putStr and main

- `putStr :: String → IO ()`

`putStr` and `main`

- `putStr :: String → IO ()`
- `putStr` takes a string as argument and returns `()`

`putStr` and `main`

- `putStr :: String → IO ()`
- `putStr` takes a string as argument and returns `()`
- Produces a side effect when the return value is extracted

`putStr` and `main`

- `putStr :: String → IO ()`
- `putStr` takes a string as argument and returns `()`
- Produces a side effect when the return value is extracted
- The side effect is that of printing the string on screen

`putStr` and `main`

- `putStr :: String → IO ()`
- `putStr` takes a string as argument and returns `()`
- Produces a side effect when the return value is extracted
- The side effect is that of printing the string on screen
- `main :: IO ()`

`putStr` and `main`

- `putStr :: String → IO ()`
- `putStr` takes a string as argument and returns `()`
- Produces a side effect when the return value is extracted
- The side effect is that of printing the string on screen
- `main :: IO ()`
- `main` is always of type `IO a`

`putStr` and `putStrLn`

- `putStr "Hello world!"` prints the string on the screen

`putStr` and `putStrLn`

- `putStr "Hello world!"` prints the string on the screen
- `putStrLn "Hello world!"` prints the string and a newline (`'\n'`) on the screen

`putStr` and `putStrLn`

- `putStr "Hello world!"` prints the string on the screen
- `putStrLn "Hello world!"` prints the string and a newline (`'\n'`) on the screen
- `putStrLn str` is equivalent to `putStr (str ++ "\n")`

Chaining actions

- We use the command **do** to chain multiple actions

Chaining actions

- We use the command **do** to chain multiple actions
- Output type of a **do** block is the same as that of the last action

Chaining actions

- We use the command `do` to chain multiple actions
- Output type of a `do` block is the same as that of the last action
- `do` makes the actions take effect in sequential order, one after the other

Chaining actions

- We use the command `do` to chain multiple actions
- Output type of a `do` block is the same as that of the last action
- `do` makes the actions take effect in sequential order, one after the other
- Use indentation or braces

```
main = do
    putStrLn "Hello!"
    putStrLn "What's your name?"

main = do {
    putStrLn "Hello!";
    putStrLn "What's your name?";
}
```

Chaining actions

- Actions can occur inside `let`, `where`, `if-then-else` etc.

```
main = let fibs = 0:1:zipWith (+) fibs (tail fibs)
    in do
        putStrLn (show fibs !! 5)
        putStrLn (show fibs !! 10)

main = do
    act1
    act2 where
        act1 = putStr "Hello, "
        act2 = putStrLn "world!"
```

More actions

- `print :: Show a ⇒ a → IO ()`

More actions

- `print :: Show a ⇒ a → IO ()`
  - Output a value of any printable type to the standard output (screen), and add a newline

More actions

- `print :: Show a ⇒ a → IO ()`
  - Output a value of any printable type to the standard output (screen), and add a newline
- `putChar :: Char → IO ()`

More actions

- `print :: Show a ⇒ a → IO ()`
  - Output a value of any printable type to the standard output (screen), and add a newline
- `putChar :: Char → IO ()`
  - Write the character argument to the screen

More actions

- `print :: Show a ⇒ a → IO ()`
  - Output a value of any printable type to the standard output (screen), and add a newline
- `putChar :: Char → IO ()`
  - Write the character argument to the screen
- `getLine :: IO String`

More actions

- `print :: Show a ⇒ a → IO ()`
  - Output a value of any printable type to the standard output (screen), and add a newline
- `putChar :: Char → IO ()`
  - Write the character argument to the screen
- `getLine :: IO String`
  - Read a line from the standard input and return it as a string

More actions

- `print :: Show a ⇒ a → IO ()`
  - Output a value of any printable type to the standard output (screen), and add a newline
- `putChar :: Char → IO ()`
  - Write the character argument to the screen
- `getLine :: IO String`
  - Read a line from the standard input and return it as a string
  - The side effect of `getLine` is the consumption of a line of input

More actions

- `print :: Show a ⇒ a → IO ()`
  - Output a value of any printable type to the standard output (screen), and add a newline
- `putChar :: Char → IO ()`
  - Write the character argument to the screen
- `getLine :: IO String`
  - Read a line from the standard input and return it as a string
  - The side effect of `getLine` is the consumption of a line of input
  - The return value is a string

More actions

- `print :: Show a ⇒ a → IO ()`
  - Output a value of any printable type to the standard output (screen), and add a newline
- `putChar :: Char → IO ()`
  - Write the character argument to the screen
- `getLine :: IO String`
  - Read a line from the standard input and return it as a string
  - The side effect of `getLine` is the consumption of a line of input
  - The return value is a string
- `getChar :: IO Char`

More actions

- `print :: Show a ⇒ a → IO ()`
  - Output a value of any printable type to the standard output (screen), and add a newline
- `putChar :: Char → IO ()`
  - Write the character argument to the screen
- `getLine :: IO String`
  - Read a line from the standard input and return it as a string
  - The side effect of `getLine` is the consumption of a line of input
  - The return value is a string
- `getChar :: IO Char`
  - Read the next character from the standard input

Binding

- `getLine` is of type `IO String`

Binding

- `getLine` is of type `IO String`
- Is there a way to use the return value?

Binding

- `getLine` is of type `IO String`
- Is there a way to use the return value?
- We need to bind the return value to an object of type `String` and use it elsewhere

Binding

- `getLine` is of type `IO String`
- Is there a way to use the return value?
- We need to bind the return value to an object of type `String` and use it elsewhere
- The syntax for binding is ←

```
main = do
    putStrLn "Please type your name!"
    n ← getLine
    putStrLn ("Hello, " ++ n)
```

Binding

- The syntax for binding is ←

```
main = do
    putStrLn "Please type your name!"
    n ← getLine
    putStrLn ("Hello, " ++ n)
```

Binding

- The syntax for binding is ←

```
main = do
    putStrLn "Please type your name!"
    n ← getLine
    putStrLn ("Hello, " ++ n)
```

- **Wrong usage** – `putStrLn ("Hello" ++ getLine)`

Binding

- The syntax for binding is ←

```
main = do
    putStrLn "Please type your name!"
    n ← getLine
    putStrLn ("Hello, " ++ n)
```

- **Wrong usage** – `putStrLn ("Hello" ++ getLine)`
- `getLine` is not a string

Binding

- The syntax for binding is ←

```
main = do
    putStrLn "Please type your name!"
    n ← getLine
    putStrLn ("Hello, " ++ n)
```

- **Wrong usage** – `putStrLn ("Hello" ++ getLine)`
- `getLine` is not a string
- It is an action that has a return value of type `String`

Binding

- The syntax for binding is ←

```
main = do
    putStrLn "Please type your name!"
    n ← getLine
    putStrLn ("Hello, " ++ n)
```

- **Wrong usage** – `putStrLn ("Hello" ++ getLine)`
- `getLine` is not a string
- It is an action that has a return value of type `String`
- The return value has to be **extracted** before use

IO example, repetition

- Read a line and print it out ten times

IO example, repetition

- Read a line and print it out ten times
- Use recursion

```
main = do
    inp ← getLine
    printOften 10 inp

printOften :: Int → String → IO ()
printOften 1 str = putStrLn str
printOften n str = do
    putStrLn str
    printOften (n-1) str
```

IO example, repetition

- How do we define `printOften 0 str`?

IO example, repetition

- How do we define `printOften 0 str`?
- Can we just define it to be `()`?

IO example, repetition

- How do we define `printOften 0 str`?
- Can we just define it to be `()`?
- But then the output would be of type `()`, not `IO ()`

IO example, repetition

- How do we define `printOften 0 str`?
- Can we just define it to be `()`?
- But then the output would be of type `()`, not `IO ()`
- Need a way to promote `()` to an object of type `IO ()`

IO example, repetition

- How do we define `printOften 0 str`?
- Can we just define it to be `()`?
- But then the output would be of type `()`, not `IO ()`
- Need a way to promote `()` to an object of type `IO ()`
- Achieved by the `return` function

IO example, repetition

- How do we define `printOften 0 str`?
- Can we just define it to be `()`?
- But then the output would be of type `()`, not `IO ()`
- Need a way to promote `()` to an object of type `IO ()`
- Achieved by the `return` function
- If `v` is a value of type `a`, `return v` is of type `IO a`

IO example, repetition

- How do we define `printOften 0 str`?
- Can we just define it to be `()`?
- But then the output would be of type `()`, not `IO ()`
- Need a way to promote `()` to an object of type `IO ()`
- Achieved by the `return` function
- If `v` is a value of type `a`, `return v` is of type `IO a`
- Not to be confused with `return` in languages like C, Java etc.

IO example, repetition

- How do we define `printOften 0 str`?
- Can we just define it to be `()`?
- But then the output would be of type `()`, not `IO ()`
- Need a way to promote `()` to an object of type `IO ()`
- Achieved by the `return` function
- If `v` is a value of type `a`, `return v` is of type `IO a`
- Not to be confused with `return` in languages like C, Java etc.
  - In imperative languages, `return` is used to return control to the caller

IO example, repetition

- How do we define `printOften 0 str`?
- Can we just define it to be `()`?
- But then the output would be of type `()`, not `IO ()`
- Need a way to promote `()` to an object of type `IO ()`
- Achieved by the `return` function
- If `v` is a value of type `a`, `return v` is of type `IO a`
- Not to be confused with `return` in languages like C, Java etc.
  - In imperative languages, `return` is used to return control to the caller
  - Here it just wraps an action around a value

IO example, repetition

- How do we define `printOften 0 str`?
- Can we just define it to be `()`?
- But then the output would be of type `()`, not `IO ()`
- Need a way to promote `()` to an object of type `IO ()`
- Achieved by the `return` function
- If `v` is a value of type `a`, `return v` is of type `IO a`
- Not to be confused with `return` in languages like C, Java etc.
  - In imperative languages, `return` is used to return control to the caller
  - Here it just wraps an action around a value
  - `x ← return e; act;` is the same as `let x = e in act;`

IO example, repetition

- Read a line and print it out ten times

```
main = do
    inp ← getLine
    printOften 10 inp

printOften :: Int → String → IO ()
printOften 0 str = return ()
printOften n str = do
    putStrLn str
    printOften (n-1) str
```

IO example, repetition – `getLine`

- Here is a possible implementation of `getLine`

```
getLine :: IO String
getLine = do
    c ← getChar
    if (c == '\n') then return ""
    else do
            cs ← getLine
            return (c:cs)
```

IO example, repetition – `getLine`

- Here is a possible implementation of `getLine`

```
getLine :: IO String
getLine = do
    c ← getChar
    if (c == '\n') then return ""
    else do
            cs ← getLine
            return (c:cs)
```

- Note the use of `return` and the recursion

Repetition and IO, another example

- Repeat an IO action n times

```
ntimes :: Int → IO () → IO ()
ntimes 0 a = return ()
ntimes n a = do
    a
    ntimes (n-1) a
```

Repetition and IO, another example

- Repeat an IO action n times

```
ntimes :: Int → IO () → IO ()
ntimes 0 a = return ()
ntimes n a = do
    a
    ntimes (n-1) a
```

- Read and print 10 lines

```
main = ntimes 10 (do
    inp ← getLine
    putStrLn inp
)
```